

RAG-vs-GraphRAG Advisor: Remaining Roadmap

Companion to `graph-vector-routing-harness-docs.md`. Section numbers (§) refer to that doc.

Where things stand

Step	Status
1. ER extraction pipeline	Mostly done: AutoSchemaKG runs and produces GraphML + CSVs from <code>.txt</code> files. Missing: PDF/CSV readers, entity resolution
2. Validate extraction	Not done: output exists but quality is unchecked (<code>inspect_graphml.py</code> is the first tool for this)
3-13	Not started

Design change to record in the main doc: extraction now uses AutoSchemaKG (`atlas_rag`) with Llama 3.1 8B 4-bit instead of GLiNER-Relex. Consequences: the schema is induced rather than passed as labels, extraction is an LLM call per chunk (slower, needs more VRAM), and §3, §4.1, §4.7 and the open questions about GLiNER need updating.

Step 1 (remainder): finish the ingestion pipeline

Goal: accept any raw corpus (§4.1), not just `.txt`.

Tasks

1. Add generic readers for PDF and CSV (format-level only, no domain logic). Output the same `{id, text, metadata}` structure that `json_parse()` already produces.
2. Fix doc IDs so skipped empty files don't leave gaps (optional).
3. Decide whether entity resolution is needed (see Step 2). If yes, build it: embedding similarity to find candidate pairs, then LLM disambiguation to confirm merges, applied to the graph before it goes to Neo4j.

Done when: a folder containing a mix of `.txt`, `.pdf` and `.csv` files runs through with no code changes.

Step 2: validate extraction on a small corpus

Goal: decide whether the extracted graph is trustworthy enough to build on.

Tasks

1. Run `inspect_graphml.py`. Record nodes, edges per node, % of nodes with 2+ neighbours, component structure, relation-type counts.
2. Hand-check about 50 random triples. Record the fraction that are correct.
3. Check duplicate and near-duplicate entities to size the entity-resolution problem.

4. If quality is poor, adjust model, chunking or batch sizes and re-run.

Done when: you have a written quality note (precision on the sampled triples, duplicate rate) and a go/no-go decision on entity resolution.

Open question to close here: how much manual QA is acceptable before the graph is populated without review (§7).

Step 3: populate Neo4j and the vector store

Goal: both backends loaded from the same corpus.

Tasks

1. Stand up Neo4j locally (Docker is simplest).
2. Write a loader from AutoSchemaKG's CSV/GraphML output into Neo4j. Inspect the CSV columns first; the loader depends on them.
3. Chunk the original documents, embed them, and store them in FAISS or Chroma. This is separate from the graph and uses the raw text.
4. Keep chunk IDs linked to document IDs so graph results and vector results can be compared on the same sources.

Done when: a Cypher query returns nodes and edges, and a vector query returns top-k chunks, both for the same corpus.

Watch for: using different corpus versions in the two backends. The step 7 comparison is invalid if they differ.

Step 4: MCP tool wrappers (§4.5)

Goal: a uniform interface the eval can call.

Tasks

1. `graph_query(cypher_or_n1_query)`: returns a subgraph or traversal result. Decide early whether natural-language input is supported (needs an LLM to produce Cypher) or Cypher only.
2. `vector_search(query, k)`: returns top-k chunks with scores.
3. Expose both through an MCP server and confirm they're discoverable and callable from a test client.

Done when: both tools can be called through MCP with a sample query and return well-formed results.

Step 5: graph structure analysis module (§4.2)

Goal: the data-only half of the recommendation.

Tasks

1. Convert the `inspect_graphml.py` metrics into a function returning a structured dict: relation density, average degree, % entities with 2+ relations, connected-component structure, relation-type diversity.
2. Compute from Neo4j (or the GraphML) so it works for any ingested corpus.
3. Decide how each metric maps to a lean toward RAG or GraphRAG. Record the thresholds, even if provisional.

Done when: one call returns the metrics dict for any ingested corpus.

Step 6: automatic query synthesis (§4.3, §5.1)

Goal: a per-corpus query set with no manual labeling.

Tasks

1. Sample real relation chains from the graph and have an LLM turn them into multi-hop questions.
2. Generate single-hop questions from entity or chunk text.
3. Target about 50-100 queries, split between relational and semantic.
4. Store each query with its type and its source chain or chunk, which gives you a reference answer.
5. Spot-check a sample by hand; synthesis quality depends on extraction quality.

Done when: a corpus produces a labeled query set with traceable ground truth.

Step 7: comparative eval (§4.3, §5.2)

Goal: the empirical half of the recommendation.

Tasks

1. Run every synthesized query through `graph_query` and `vector_search`.
2. Generate an answer from each retrieval result using the same LLM and prompt.
3. Score answer quality against the reference (LLM judge, with a hand-checked sample to validate the judge).
4. Compute the answer-quality delta per query and in aggregate, split by query type.
5. Record end-to-end runtime.

Done when: you have, per corpus, a table of per-query quality for both paths and an aggregate delta.

Watch for: judge bias, and queries whose reference answer came from the graph (they favor graph retrieval by construction; consider checking that vector-friendly queries aren't penalized).

Step 8: recommendation engine (§4.4)

Goal: a single output of GraphRAG / RAG-only / Hybrid with confidence and evidence.

Tasks

1. Combine the structural signal (Step 5) and empirical signal (Step 7).
2. Decide how to weight them when they disagree: simple average, or one overrides the other in defined cases.
3. Decide whether Hybrid has its own thresholds or falls out as "neither signal strongly one-sided."
4. Report disagreement between the two signals as a finding, not an error.
5. Produce the report format from §4.4 (recommendation, confidence, evidence list).

Done when: one call on an ingested corpus returns the full diagnostic report.

Step 9: FastAPI service (§4.6)

Tasks

1. `POST /ingest`: accept an upload or path, start extraction as a background job, return a `job_id`.
2. `POST /analyze`: run Steps 5-8, return the report.
3. `GET /status/{job_id}`: report progress for long-running jobs.
4. Use a job queue or background tasks, since extraction takes minutes to hours.

Done when: a corpus can be ingested and analyzed through HTTP calls only.

Step 10: containerize and deploy on Kubernetes (§4.7)

Tasks

1. Write Dockerfiles for the API and the extraction worker (GPU-capable, if the cluster supports it).
2. Write manifests for four services: `fastapi-service`, `neo4j`, `vector-store`, `extraction-worker`.
3. Configure service discovery between the API and both backends.
4. Optional: an HPA on the extraction worker.
5. Test locally first (kind or minikube) before any cloud cluster.

Watch for: an 8B model in a pod needs GPU access and enough VRAM; this is likely the hardest part of this step.

Done when: the full ingest-analyze flow works against the cluster.

Step 11: known-answer validation (§5.3)

Tasks

1. Choose the two corpora (open question, see below): one clearly low-relational, one

clearly high-relational.

2. Run the full pipeline on both.
3. Confirm the tool recommends RAG-only for the first and GraphRAG for the second.
4. Run the same corpus twice and compare recommendations (cross-corpus consistency, checks run-to-run variance).

Done when: both known cases are classified correctly, and repeat runs agree.

Step 12: data-agnosticism check

Tasks

1. Pick a third, unrelated corpus.
2. Run the full pipeline with zero code changes.
3. Note anything that needed manual intervention; each such item is a gap in the claim.

Done when: the pipeline completes and produces a sensible report with no edits.

Step 13 (optional): n8n automation (§4.8)

Tasks

1. Trigger `/analyze` on a schedule or on new-data arrival.
 2. Post the report summary to Slack or email.
 3. Compare against the previous recommendation and flag changes.
-

Decisions to make

Decision	Needed by
Replace GLiNER-Relex with AutoSchemaKG in the doc, or compare both?	Before updating the doc
Entity resolution: build or skip	End of Step 2
Cypher-only vs natural-language <code>graph_query</code>	Step 4
Two known-answer corpora	Before Step 11, but pick early since it shapes what you test on
Structural vs empirical weighting; Hybrid threshold logic	Step 8
Cost/complexity estimate in the report, or scope to architecture fit only	Step 8

Suggested order

Steps 2 → 5 (needs no Neo4j) → 3 → 4 → 6 → 7 → 8 → 9 → 11 → 10 → 12 → 13. Doing Step 11 before

Step 10 means you validate the logic locally before spending time on Kubernetes.